

Table 9-1: Comparison Operator Symbols, Meanings, and Magic Method Names

Symbol	Meaning	Magic method name
==	Equal to	<code>__eq__()</code>
!=	Not equal to	<code>__ne__()</code>
<	Less than	<code>__lt__()</code>
>	Greater than	<code>__gt__()</code>
<=	Less than or equal to	<code>__le__()</code>
>=	Greater than or equal to	<code>__ge__()</code>

To allow the `==` comparison operator to check for equality between two `Square` objects, you would write a method like this in the `Square` class:

```
def __eq__(self, oOtherSquare):
    if not isinstance(oOtherSquare, Square):
        raise TypeError('Second object was not a Square')
    if self.heightAndWidth == oOtherSquare.heightAndWidth:
        return True # match
    else:
        return False # not a match
```

When Python detects an `==` comparison where the first object is a `Square`, it calls this method in the `Square` class. Since Python is a loosely typed language (it doesn't require you to define variable types), the second parameter could be of any data type. However, in order for the comparison to work correctly, the second parameter must also be a `Square` object. We perform a check using the `isinstance()` function, which works with programmer-defined classes the same way it works with built-in classes. If the second object is not a `Square`, we raise an exception.

We then compare the `heightAndWidth` of the current object (`self`) with the `heightAndWidth` of a second object (`oOtherSquare`). This is a case where using direct access to the instance variables of two objects is perfectly acceptable, because both objects are of the same type, and therefore they must contain the same instance variables.